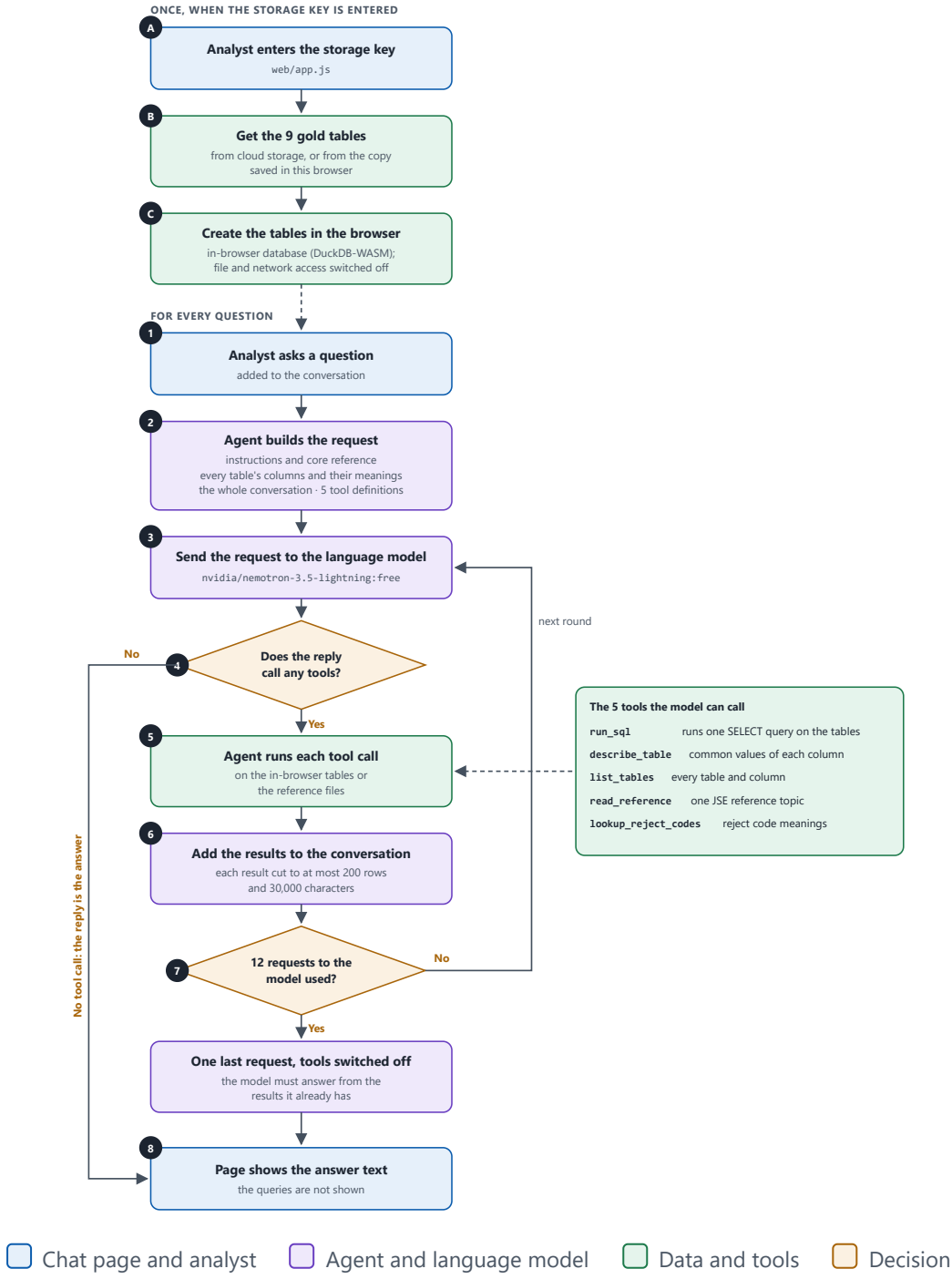
How the chat agent answers a question

The chat agent answers a question about the conformance test messages by repeating one cycle. It sends the question to a language model, runs the database queries the model writes, and returns the results to the model. It stops when the model replies with an answer instead of another query. The queries run on a copy of the gold tables held in the analyst's browser, so no data is fetched while a question is being answered.



Before the first question: loading the data

A. The analyst pastes the storage key into the sidebar of the chat page. The key gives read access to the gold tables in cloud storage.

B. `web/data.js` downloads `manifest.json`, the list of the 9 gold tables with a description of every column. It then gets the 9 tables, which are Parquet files (a compressed file format for tables). If the browser already holds a saved copy that matches the current `manifest.json`, it reads that copy. Otherwise it downloads the files from cloud storage and saves a copy for the next visit.

C. It creates the 9 tables in DuckDB-WASM, a database that runs inside the browser tab and holds the tables in the tab's memory. It then switches off the database's access to files, to the network and to its own settings. From this point a query can read those 9 tables and nothing else. The sidebar shows "Data loaded. The agent can answer questions."

Answering a question

1. The analyst types a question. The chat page adds it to the conversation, which is the list of every question and answer so far in this session.
2. `web/agent.js` builds one request with five parts: its instructions; the core of the JSE conformance reference (`web/knowledge/core.md`); every table's columns and their meanings; the whole conversation; and the definitions of the 5 tools. A tool is an action the agent can carry out on the model's behalf, such as running a query.
3. The agent sends the request to the language model `nvidia/nemotron-3.5-lightning:free`. The model is set to temperature 0, which removes most of the randomness from its replies.
4. The model's reply contains either answer text, or one or more tool calls. A tool call names a tool and gives its input, for example `run_sql` with a query the model has written. If the reply contains no tool call, the reply is the answer and the agent goes to step 8.
5. The agent runs each tool call. `run_sql` runs the model's query on the in-browser tables. The query is placed inside `SELECT * FROM ( … ) LIMIT 201`, so it can only read data and cannot return more than 201 rows. `read_reference` and `lookup_reject_codes` read the JSE reference files in `web/knowledge/`.
6. Each result is cut to at most 200 rows and 30,000 characters and added to the conversation, after the model's tool call.
7. If fewer than 12 requests have been sent to the model for this question, the agent goes back to step 3 with the longer conversation. After 12 requests it sends one last request with tools switched off. That request tells the model to answer from the results it already has and to say which parts of the question it could not finish.
8. The page shows the answer text. The tool calls and queries are not shown. The instructions tell the model to write in plain words, without SQL, table names or column names.

Example

This is an illustration of the steps, not a recorded run. An analyst asks: "Which reject codes did CompID GJEN09 receive in August 2026, and what do they mean?" (GJEN09 is one of the fictitious CompIDs.)

1. Round 1: the model replies with one `run_sql` call. Its query counts the rows of `fact_message` for CompID GJEN09 dated in August 2026 that have a reject code, and joins `dim_reject_code` to add each code's meaning.
2. The agent runs the query on the in-browser tables, which takes a fraction of a second, and adds the rows to the conversation.
3. Round 2: the model replies with text that lists each reject code, how many times it was received and what it means. The page shows that text.

A question that needs the JSE test rules, such as whether a test was passed, usually takes more rounds. The model first calls `read_reference` for the `tests` topic, then queries the messages.

Where the time is spent

Step	Time measured	Measured how
Loading the data, first visit (download from cloud storage)	33.6 s	Headless Edge on the project computer, 01/11/2026
Loading the data, later visits (saved copy)	10.8 s	Same test, second load
One query on the in-browser tables	23 to 86 ms	Four typical queries, same test
One request to the language model	Not measured	Needs an OpenRouter key on the test computer

A query takes less than a tenth of a second, so the time an analyst waits for an answer is mostly the time spent waiting for the language model's replies. Each round adds one model reply. Fewer rounds per question give faster answers. For that reason every table's columns are sent with the first request, so the model does not spend a round asking for them.